

All Projects — Swanand Kadam

Single-file reference covering all 6 projects: architecture, decisions, technology comparisons, and interview Q&A.

Pair with `SYSTEM_DESIGN_PATTERNS.md` (general patterns) and `AI_AGENT_SYSTEM_DESIGN.md` (AI-specific patterns).

Project Index

#	Project	What it is	Scale / Constraint	Key stack
1	CodeMas	Secure real-time coding exam platform	10K concurrent students	Vue 3 · Django · SQS FIFO · Lambda · Postgres
2	Kalaam	Programming language in Hindi/Marathi/Bengali/Telugu/Odia	Mobile-first, offline-first	Vue 2 · Quasar PWA · CodeMirror · pure JS interpreter · npm
3	o.1% DEV	Sovereign AI ed-tech (deep CS fundamentals)	Fully local inference, EU-AI-Act compliance	React 19 · FastAPI · Hermes · LangGraph · Ollama/vLLM · pgvector
4	stringy-core	Zero-dependency JS string utility library	50+ functions, tree-shakeable, open-source	ESM · Jest · Intl API · Husky + lint-staged
5	Munshi	Sovereign AI agent for GST reconciliation	Financial data never leaves the machine	Hermes · MCP · FastAPI · Ollama · Python Decimal · SQLite
6	Trade Compliance Researcher	Multi-agent tariff and trade regulation research	Two independent agents, no HITL today	Hermes · MCP · Ollama · Docker Compose

1. CodeMas — Secure Coding Exam Platform

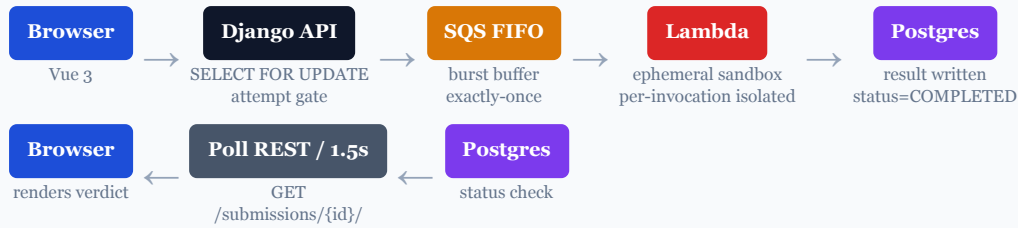
Quick Reference

Field	Detail
Platform	Real-time coding assessment — secure, proctored online exams
My Role	Sole engineer — architecture, backend, frontend, infra, deployment
Scale	10,000 concurrent students; ~10K submissions/sec at exam-deadline bursts
Submission SLA	Result delivered to browser within 10s (p95)
Result delivery	Client-side polling — frontend polls Postgres result via API every 1.5s
Plagiarism	Two-phase post-exam pipeline; 19× improvement in catch rate
AI Features	5 LLM features; zero AI on the submission critical path
Stack	Vue 3 + Vite · Django 4.2 + DRF · Simple JWT · SQS FIFO · Lambda · Postgres · Redis (cache only)

Architecture Overview

1. Student submits code → Django API checks attempt count with `SELECT FOR UPDATE` (prevents double-execution race condition) → INSERTs submission record with status `PENDING` → sends message to SQS FIFO with `MessageGroupId = student_id` → returns 201 immediately.
2. Lambda is triggered by SQS. Executes student code in an ephemeral, resource-limited Lambda environment. Writes result (verdict, output, score) to Postgres with status `COMPLETED`. Invocation is acknowledged; SQS message deleted.
3. Browser polls `GET /submissions/{id}/` every 1.5s. When status is `COMPLETED`, renders verdict. SLA: result in browser within 10s p95.
4. Trainer dashboard: single fetch on mount, Redis-cached (TTL 10s). Celery handles the 5 async AI features (rubric scoring, hint generation, exam generation, plagiarism explanation, performance summary) — none on the submission critical path.
5. Plagiarism runs post-exam. Django `pre_save` signal on Exam detects `is_active True → False`. Phase 1 (Behavioural): $O(N)$ — paste ratio, speed vs baseline, tab switches, attempt surprise. Flags `K` suspects. Phase 2 (Similarity): $O(K \times N)$ TF-IDF + cosine — `K` suspects compared against full cohort. Flags pairs above 0.80 similarity.

SUBMISSION FLOW – CRITICAL PATH (P95 < 10 S)

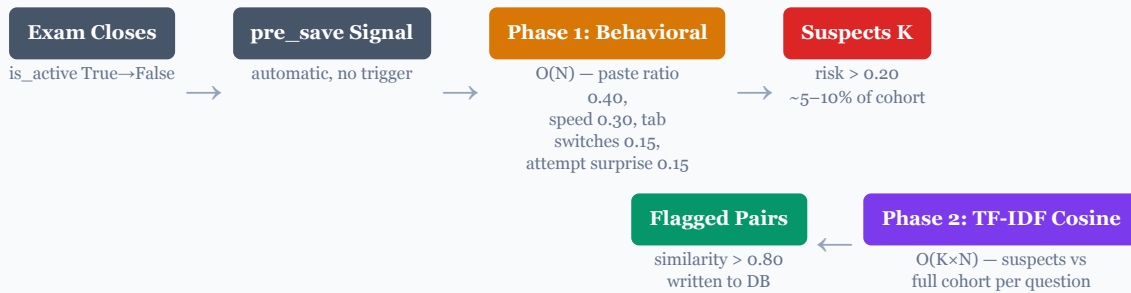


SUPPORTING SERVICES (OFF CRITICAL PATH)



Redis is NOT on the submission critical path after the Lambda migration. In the original Celery architecture, Redis pub/sub delivered results end-to-end (true push). Now: SQS owns the queue, Lambda owns execution, Redis stays for async tasks + cache.

PLAGIARISM – TWO-PHASE POST-EXAM PIPELINE



$N^2=2$ billion comparisons at 10K students. Behavioral pre-filter reduces to $K \times N \approx 5$ million. $19\times$ improvement in catch rate.

Key Engineering Decisions

Decision	Chosen	Alternatives	Why
Execution sandbox	AWS Lambda	Docker on EC2, Fargate	Per-invocation isolation, zero fleet management, per-ms billing matches burst traffic
Queue	SQS FIFO	Kafka, RabbitMQ	Exactly-once per <code>MessageGroupId</code> , managed, no replay needed, burst auto-scaling
Result delivery	Client polling every 1.5s	SSE, WebSocket	After Lambda migration, SSE was polling Postgres on client's behalf — no real advantage. Polling is honest and stateless.
Attempt gate	<code>SELECT FOR UPDATE</code> (Postgres)	Application lock, Redis INCR	Atomic read-increment-write; prevents lost-update race condition without a separate lock service
Plagiarism order	Behavioural first ($O(N)$) then Similarity ($O(K \times N)$)	Similarity first ($O(N^2)$)	$N=10K$ students, 20 questions → exhaustive pairwise = 2 billion comparisons; behavioural pre-filter drops K to ~5–10%
Plagiarism trigger	<code>pre_save</code> signal on Exam	Cron job, admin button	Fires automatically on state transition; no manual trigger

Technology Comparisons

SQS FIFO vs Kafka vs RabbitMQ

	SQS FIFO	Kafka	RabbitMQ
Delivery	Exactly-once per group	At-least-once (need idempotent consumers)	At-least-once (manual ack)
Replay	No	Yes	No
Multiple consumers	One consumer per queue	Many independent consumer groups	Flexible routing via exchanges
Throughput	3,000 msg/s per group	Millions/s	High, cluster-dependent
Ops burden	Zero — managed	High — cluster, replication	Medium — cluster

For CodeMas: SQS FIFO. Exactly-once prevents double execution. Managed means no cluster at 3am during live exam. No replay or fan-out needed.

Lambda vs Celery+Docker vs Fargate

	Lambda	Celery + Docker	Fargate
Isolation	Per-invocation ephemeral	Container (shared host kernel)	Container-level
Cold start	<100ms warm / ~500ms cold	Instant (persistent workers)	10–30s (task startup)
Resource limits	Hard — enforced by AWS	Soft — must configure manually	Hard — task-level
Billing	Per 1ms of execution	Workers run 24/7	Per vCPU-second of task
Scale	Instant, per-invocation	Capacity planning required	Slower than Lambda

For CodeMas: Lambda. Per-invocation model matches burst exam traffic. Hard limits prevent runaway student code. No persistent state across invocations.

Top Interview Q&A

Q: Why did you remove SSE?

After the Lambda migration, SSE had no real-time event to subscribe to. In the original Redis pub/sub architecture, Lambda published to a channel and Django SSE was subscribed — true server push. After Lambda started writing results to Postgres, the SSE handler had to poll Postgres every 500ms on the client's behalf. A server polling Postgres every 500ms and forwarding via SSE is functionally identical to the client polling a REST endpoint every 1.5s — but with the overhead of a persistent HTTP connection on a Gunicorn worker and Nginx timeout configuration. Client polling is the honest mechanism. If we need true push, the right investment is API Gateway WebSocket where Lambda explicitly notifies the connection manager.

Q: Walk me through `SELECT FOR UPDATE` — what race condition does it prevent?

Without the lock: two browser tabs submit simultaneously. Both read `attempt_count = 2` (limit 3), both see $2 < 3$, both proceed. Both increment and write 3. The counter correctly shows 3 but two Lambda invocations run — student consumed one attempt but got two executions. `SELECT FOR UPDATE` holds the row lock from read through commit. The second transaction blocks until the first commits, then reads the already-incremented 3, sees it equals the limit, and rejects. Classic lost-update problem. Postgres row lock is the correct tool; Redis INCR would work but adds a dependency for a problem that Postgres already solves.

Q: Why behavioral signals before similarity? Why not run similarity first since it's more definitive?

Similarity is $O(N^2)$ exhaustively. At $N=10,000$ students $\times$ 20 questions, that's 2 billion comparisons — minutes of compute. Behavioral scoring reads each student's event log once — $O(N)$, runs in seconds. It produces suspect set K (typically 5–10% of cohort). Phase 2 is $O(K \times N)$ — $K=500$ suspects vs 10,000 full cohort = 5 million comparisons per question. Order of magnitude faster, and behavioral signals are reliable enough as a coarse filter. Paste ratio 0.9 + 30-second submission on a hard problem has a high prior for cheating. Similarity confirms and identifies the source.

Q: Why is zero AI on the submission critical path?

AI features are async Celery tasks — hint generation, rubric scoring, exam generation. If the LLM API is slow or down, it affects optional features, not the exam grade or submission receipt. The exam SLA (result in browser within 10s) cannot depend on a non-deterministic external service. AI is best-effort; the critical path is deterministic.

Q: How do you handle the burst at exam close when many students submit simultaneously?

SQS absorbs the burst without configuration — it queues at millions/sec. Lambda scales concurrently per-message: 10,000 submissions → 10,000 Lambda invocations, each isolated. Provisioned Concurrency is pre-warmed before each exam based on registered student count, eliminating cold starts for the burst. The Django API is stateless — Gunicorn with gevent workers handles thousands of concurrent poll requests because each yields during the Postgres read.

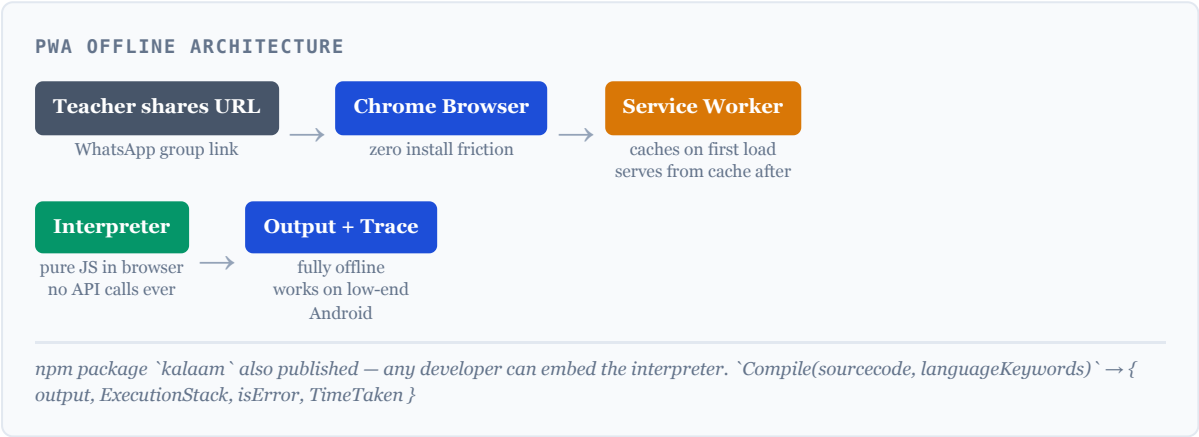
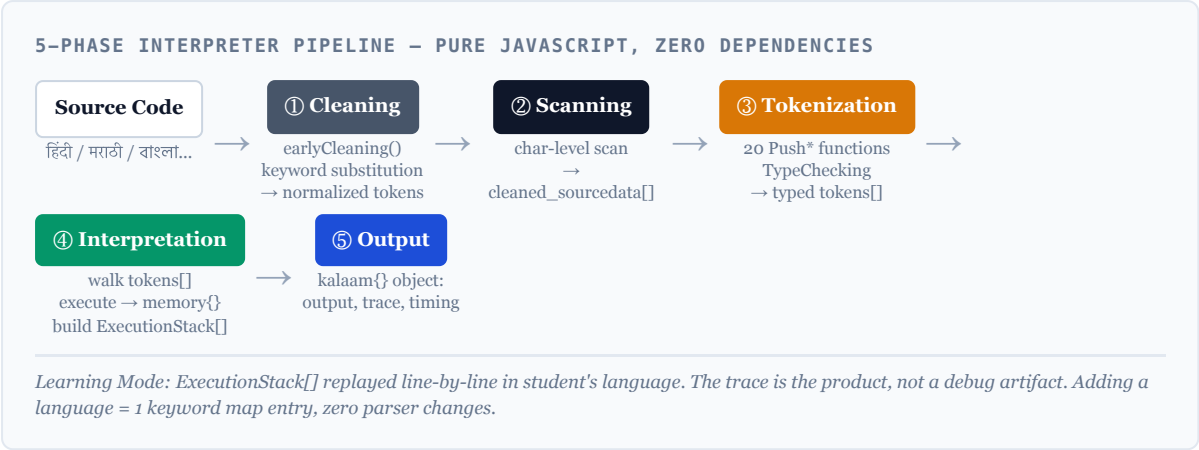
2. Kalaam — Programming Language in Indian Languages

Quick Reference

Field	Detail
Platform	Mobile-first offline PWA + npm package
Languages	Hindi, Marathi, Bengali, Telugu, Odia
Audience	Tier-3 city students aged 14–18; no laptop, intermittent internet
npm package	kalaam v2.3.3, zero runtime dependencies
Public API	<code>Compile(sourcecode, languageKeywords) → { output, ExecutionStack, isError, TimeTaken }</code>
Test coverage	90–95%, enforced by Jest
Stack	Vue 2 · Quasar (PWA) · CodeMirror (custom syntax mode) · pure JS interpreter · Jest

Architecture Overview

- Interpreter is published as the `kalaam` npm package with zero runtime dependencies. Public API: `Compile(sourcecode, languageKeywords) → { output, ExecutionStack, isError, TimeTaken }`.
- Five-phase pipeline: (1) Cleaning — `earlyCleaning()` + keyword substitution maps language keywords to normalized tokens; (2) Scanning — character-level scan → `cleaned_sourcedata[]`; (3) Tokenization — 20 `Push*` functions → typed `tokens[]`; (4) Interpretation — walks `tokens[]`, executes against `memory{}`, builds `ExecutionStack[]`; (5) Output — returns `kalaam{}` object.
- Frontend (`kalaamlang.in`): static PWA, service-worker cached. All interpreter code runs client-side. Zero API calls after initial load.
- Learning Mode: every operation appends to `ExecutionStack[]`. UI replays the stack line-by-line in the student's language, showing how the interpreter evaluated each step. This is the product, not a debug artifact.
- Adding a new language = one keyword map entry. No parser changes. The tokenizer works on normalized tokens, not raw language keywords.



Key Engineering Decisions

Decision	Chosen	Alternative	Why
Execution model	Tree-walking interpreter	Bytecode VM, native compiler	Learning Mode (ExecutionStack) is native to a tree-walking interpreter — no instrumentation needed; bytecode would require explicit tracing
Distribution	npm package	Bundled with frontend only	Any developer can embed the interpreter in their own product; separates engine from UI
Runtime dependencies	Zero	lodash, parsers, etc.	Works in any JS environment; no transitive dependency risk; offline installs cleanly
Module format	ESM	CommonJS	Tree-shakeable; consumers import only what they use
Deployment	PWA	React Native, native Android	Install barrier for tier-3 demographic; URL access via WhatsApp group link is zero friction
Offline strategy	Service worker + Cache API	Periodic sync	First load caches everything; every subsequent visit is fully offline, no partial availability

Technology Comparisons

Interpreter vs Compiler vs Transpiler vs Bytecode VM

	Tree-walking interpreter	Bytecode VM	Native compiler	Transpiler
Build step	None	Yes (compile to bytecode)	Yes (compile to machine code)	Yes (compile to another language)
Execution speed	Slowest	Fast	Fastest	Depends on target runtime
Execution trace	Native — every step is visible	Requires instrumentation	Requires debug symbols	Via source maps
Examples	Kalaam, early Ruby	Python (CPython), JVM	C, Rust, Go	TypeScript, Babel

For Kalaam: tree-walking interpreter. Learning Mode requires a step-by-step trace as a first-class feature. The interpreter produces it for free; any other model would require explicit instrumentation. Student programs are small — execution speed is not a constraint.

PWA vs React Native vs Native App

	PWA	React Native	Native (Android/iOS)
Installation	URL → browser → Add to Home Screen	Play Store download (~50MB)	Play Store download (~100MB+)
Install barrier	None	Meaningful	High
Offline	Service worker + Cache API	AsyncStorage + offline-capable	Full device storage
Device API	Limited	Full native API	Full native API
Update mechanism	Deploy to web server — instant	Play Store review cycle (days)	Play Store review cycle

For Kalaam: PWA was not a trade-off — it was the natural architecture. The teacher pastes a URL in a WhatsApp group; the student opens it in Chrome. First load caches everything. The interpreter is pure JS running in the browser. PWA = zero install friction + fully offline + instant update.

Top Interview Q&A

Q: What kind of language implementation is Kalaam? Why a tree-walking interpreter?

Kalaam is a tree-walking interpreter. It walks tokens at runtime with no ahead-of-time compilation step. The reason is Learning Mode: every operation appends to `ExecutionStack[]`, which the UI replays line-by-line in the student's mother tongue showing exactly how the interpreter evaluated the program. This trace is the product's primary teaching feature, not a debug artifact. A bytecode VM or compiler could produce a trace, but you'd add explicit instrumentation — the tree-walking interpreter does it naturally as a side-effect of execution. Performance is irrelevant: student programs are 10–50 lines.

Q: Why a zero-dependency npm package?

The interpreter is distributed as `kaalaam` on npm to let any developer embed it in their own educational product without inheriting Kalaam's full dependency tree. Zero runtime dependencies

means: any JS environment (Node, Deno, browser), no transitive package risk (no left-pad incident, no supply chain concern), and it installs cleanly on slow connections. Dev dependencies (Jest, Babel) are present for testing but never installed by consumers.

Q: How does adding a new language work?

One keyword map entry. The tokenizer doesn't see raw language keywords — it sees normalized tokens after the keyword substitution phase. Adding Gujarati = add one object mapping `{ "જર": "if", "છાપા": "print", ... }` to the language registry. The scanner, tokenizer, interpreter, and Learning Mode trace all work unchanged. This is the zero-parser-change promise.

Q: Why did you give it a TEDx talk?

The hypothesis I was arguing: the bottleneck to programming adoption in India's tier-3 cities is not infrastructure or hardware — it's language. If a 14-year-old in Nashik can't read English, "if (condition)" is noise. `यदि (अट)` is immediately intelligible. The TEDx talk was about making the zeroth step — understanding what a conditional is — accessible in the student's own language before they encounter English syntax.

3. 0.1% DEV — Sovereign AI Ed-Tech Platform

Quick Reference

Field	Detail
Platform	Deep CS fundamentals video courses with AI tutoring
Tagline	"Become 'THAT' Developer"
Courses	8 — Build a Programming Language, Build a Real-Time System, JS Engine Internals, etc.
Stack	React 19 · TypeScript · Vite · FastAPI · Hermes agent · LangGraph · pgvector · Ollama (dev) · vLLM (prod)
Auth	Firebase Auth + firebase-admin JWT
Payments	Razorpay (Indian payment gateway)
Sovereignty	Generation: 100% local. Retrieval embedding: OpenAI text-embedding-3-small (one remaining external call)
AI features	Sovereign RAG tutor · GEPA self-evolving SOUL · QuizMe (HITL) · Artifact generation · RAGAS eval

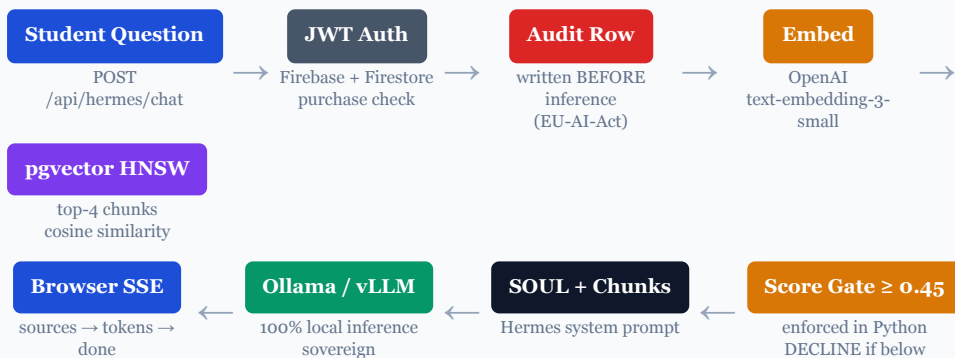
Architecture Overview

- 1. **Two-engine design:** Hermes (ReAct agent) for open-ended AI decisions; LangGraph (StateGraph) for known step sequences with HITL checkpoints. Rule: agent when the AI decides its path; workflow when every step is known.
- 2. **RAG Tutor** (`POST /api/hermes/chat`): Firebase JWT validation → audit row written *before* inference (EU-AI-Act) → load student profile + last 3 turns → embed question (OpenAI) → HNSW search (pgvector, top-4 chunks) → grounding gate (score ≥ 0.45, enforced in code) →

inject chunks into SOUL system prompt → stream from Ollama/vLLM via SSE (sources event → token events → done event) → record turn.

3. **Grounding gate:** if max cosine similarity < 0.45, model is never called. Deterministic DECLINE returned in Python. No prompt instruction — structural enforcement.
4. **GEPA:** offline pipeline — eval current SOUL on test set → reflect on 3 weakest cases → propose revised SOUL → re-eval → Pareto gate (avg improves AND no case < 2) → human approves → write SOUL.md with SHA fingerprint.
5. **QuizMe (LangGraph):** plan_objectives → interrupt (HITL approve) → generate_quiz → interrupt (submit) → [loop per objective] → summarize. MemorySaver checkpoint — survives server restarts. Student can pause mid-quiz for days.
6. **RAGAS:** sovereign judge pointed at local model. Metrics: faithfulness + LLM context precision. Feeds the grounding signal that GEPA optimizes.

RAG TUTOR PIPELINE — GROUNDED, AUDITED, SOVEREIGN



If score < 0.45: model is NEVER called — deterministic DECLINE returned in code, not a prompt instruction. The one external call: OpenAI embeddings (swappable in one line).

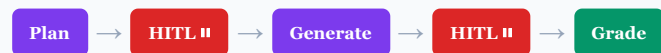
TWO-ENGINE DESIGN — HERMES VS LANGGRAPH

Hermes (ReAct Agent)



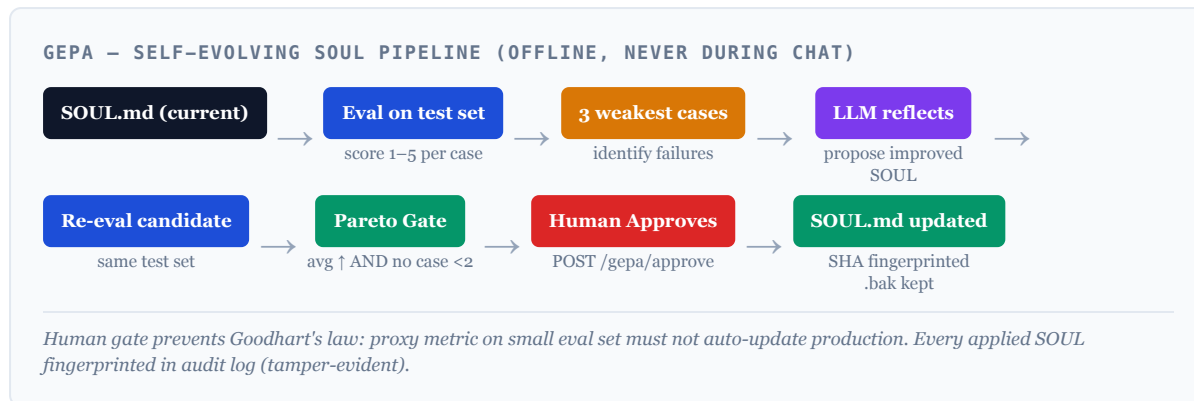
Open-ended — AI decides next step. Tool path unknown in advance.
→ **RAG Tutor**

LangGraph (StateGraph)



Known steps + HITL checkpoints. MemorySaver — survives server restarts.
→ **QuizMe · Artifact Gen**

Rule: enumerate every step before execution → LangGraph. AI must decide its own path → Hermes. Wrong pairing: rigid agents or non-deterministic workflows.



Key Engineering Decisions

Decision	Choice	Alternative	Why
Two orchestration engines	Hermes + LangGraph	Single framework	Agents for open-ended, workflows for known steps — mismatching causes rigid agents or non-deterministic workflows
RAG generation	Local Ollama / vLLM	GPT-4 API	Sovereign inference — course content and student questions never leave the server
Grounding gate	Score ≥ 0.45 , enforced in code	Prompt instruction	Structural guarantee — model cannot hallucinate on low-confidence retrieval because it is never called
GEPA human approval	Required before applying	Auto-deploy	Defense against Goodhart's law — proxy metric on small eval set should not auto-update production
RAGAS judge	Local model	External evaluation API	Zero external calls for quality measurement — sovereign quality loop
HITL	LangGraph <code>interrupt()</code>	Custom polling	Built-in checkpointing; suspend/resume across HTTP requests and server restarts
Streaming	SSE	WebSocket, REST polling	Progressive token delivery; server → client only; works through standard HTTP infrastructure
Prices	Integer paise	Float rupees	Float precision errors in financial arithmetic
Audit timing	Write before inference	Write after	Tamper-evident; record exists even if process crashes between write and inference
pgvector vs dedicated vector DB	pgvector (Postgres)	Pinecone, Chroma	Small corpus; ACID consistency with purchase data; one fewer managed service; data stays in-house

Technology Comparisons

RAG vs Fine-tuning vs Long Context

	RAG	Fine-tuning	Long Context
Knowledge updates	Instant (update vector store)	Full retraining required	Instant (swap documents)
Source attribution	Native (retrieved chunks = citations)	None — knowledge baked in	Native
Hallucination control	Grounding gate enforces relevance	Model can hallucinate fine-tuned facts	Long context increases distraction
Best for	Dynamic knowledge, citations needed, large corpus	Stable behavior, persona, domain vocabulary	Small stable corpus, simple queries

For o1dev: RAG. Course transcripts update as new courses ship. Students must see citation sources (Socratic teaching). Corpus grows unboundedly. Grounding gate is a structural safety valve fine-tuning cannot provide.

pgvector vs Pinecone vs Chroma

	pgvector	Pinecone	Chroma
Location	Same Postgres DB	External managed service	Local process or server
Transactions	ACID	No	No
Scale ceiling	Single Postgres node (tens of millions)	Billions, managed	Thousands to low millions
Data sovereignty	Your infrastructure	Pinecone's cloud	Your infrastructure

For o1dev: pgvector. Small corpus, transactional consistency with purchase records, sovereign, no extra service.

SSE vs WebSocket for LLM token streaming

	SSE	WebSocket
Direction	Server → Client	Bidirectional
Proxy/LB compatibility	Excellent (standard HTTP)	Requires explicit WS support
Auto-reconnect	Built into browser EventSource	Application-level
Use WebSocket when	Client needs to send messages mid-stream (stop, edit)	—

For o1dev: SSE. LLM streaming is always server→client. Client never interrupts mid-generation. Works through standard load balancers.

Top Interview Q&A

Q: Walk me through the full RAG tutor pipeline.

Request hits FastAPI at :8080. (1) Firebase JWT validated, Firestore checked for course purchase. (2) Audit row inserted: `{uid, question, model, soul_hash, ts, answer: null}` — before anything else. (3) Student profile and last 3 turns loaded into system prompt. (4) Question embedded via OpenAI

text-embedding-3-small. (5) HNSW ANN search — top-4 chunks by cosine similarity. (6) Grounding gate: if max score ≥ 0.45 , chunks injected into SOUL prompt and Ollama/vLLM called. Response streams as SSE: `event:sources` (retrieved chunks) $\rightarrow$ `event:token` (per-token) $\rightarrow$ `event:done`. React renders SourceCards as clickable deep-links to the video timestamp. If score < 0.45 and no student consent for general knowledge, deterministic DECLINE returned — model never called.

Q: How did you pick 0.45 as the grounding threshold?

Empirically. Built a calibration set: 50 in-scope questions (answers in course transcripts) and 50 out-of-scope questions. Ran retrieval on all 100 and plotted score distributions. In-scope clustered above 0.55; out-of-scope clustered below 0.40. Set threshold at 0.45 — midpoint of the gap — zero false positives, zero false negatives on the calibration set. Validated on a held-out 30-question set. The threshold is a `RAG_THRESHOLD` environment variable; it can be tuned per course or per use case without code changes.

Q: What is GEPA? Walk me through the self-improvement loop.

GEPA = Generative Evaluation and Prompt Adaptation. Runs offline, never during chat. Pipeline: (1) Evaluate current SOUL.md on a fixed QA test set, score each 1–5 on faithfulness/helpfulness/groundedness. (2) Identify 3 weakest cases. (3) Prompt local model: "Here is the SOUL and here are the 3 worst cases. Propose a revised SOUL that addresses them." (4) Re-evaluate candidate on same test set. (5) Pareto gate in code: candidate must score higher on average AND score ≥ 2 on every individual case — no regression allowed. (6) Human reviews diff, approves via API. (7) New SOUL applied with SHA fingerprint recorded in audit log; previous kept as SOUL.md.bak. Human gate exists because GEPA optimizes a proxy metric on a small set — without it, the model could game the eval set (Goodhart's law) without improving on novel inputs.

Q: Why two orchestration engines? Why not just Hermes for everything?

The engines solve different problems. Hermes is a ReAct agent — the model decides its next step based on what it observes. For the RAG tutor, "answer this question" might require one retrieval or three depending on what the model finds. The number of steps is not known in advance. Hermes is designed for this. QuizMe is different: plan objectives $\rightarrow$ HITL approve $\rightarrow$ generate quiz $\rightarrow$ HITL submit $\rightarrow$ grade $\rightarrow$ summarize. Every step is known before execution. Giving this to Hermes means the agent might skip the approval step because it "decides" the plan looks fine, or ask a clarifying question instead of generating. LangGraph's StateGraph guarantees the defined steps in order with interrupt points at specific nodes. Decision rule: if you can enumerate every step before execution $\rightarrow$ LangGraph. If the AI must decide its own path $\rightarrow$ Hermes.

Q: RAG vs fine-tuning — why not fine-tune on the course content?

RAG for knowledge, fine-tuning for behavior. Course transcripts update when new courses ship — fine-tuning requires a full retraining cycle. Students need to see which transcript excerpt supports the answer — fine-tuning bakes knowledge invisibly into weights with no attribution. The grounding gate (0.45) provides a structural safety valve against hallucination — fine-tuning cannot give you this because the model can hallucinate its own fine-tuned knowledge confidently. Fine-tuning would be the right tool if we wanted to change the tutor's communication style or enforce a response format — stable behavior changes, not knowledge changes.

4. stringy-core — JavaScript String Utility Library

Quick Reference

Field	Detail
Package	<code>stringy-core</code> on npm
Functions	50+ across 9 modules
Dependencies	Zero runtime dependencies
Module format	ESM (<code>"type": "module"</code>)
Test coverage	Jest; assertion-based suite
Export patterns	Named imports (tree-shakeable) + namespace <code>_s</code> (convenience)
Pre-commit	Husky + lint-staged → ESLint + Prettier
Known bug	<code>isPalindrome</code> returns string instead of boolean

Architecture Overview

1. Nine modules organized by concern: `textCaseManipulation`, `textCleaning`, `textFormatting`, `textMaskingAndSecurity`, `textMetadataAndExtraction`, `textAnalysisAndValidation`, `textTransformations`, `textSpecializedOperations`, `textGeneration`.
2. All formatting functions (`formatNumber`, `formatCurrency`, `formatDate`, `formatRelativeTime`) delegate to the `Intl` API — zero locale data shipped inside the package; the runtime provides it.
3. Two export patterns: named imports (`import { maskEmail } from 'stringy-core'`) for tree-shaking; namespace (`import { _s } from 'stringy-core'`) for convenience. Both documented.
4. Pre-commit hook via Husky + lint-staged: ESLint + Prettier run only on staged files. Fast, consistent.
5. Jest + Babel for testing: Babel transpiles ESM → CJS so Jest (a CJS runner) can execute the test files. Consumer code uses native ESM; the Babel config is dev-only.
6. Contribution stubs: `// CONTRIBUTION_STUB` functions with complete signatures and JSDoc but `throw Error('Not implemented')` — they signal to community contributors exactly what to build.

9-MODULE ARCHITECTURE – 50+ FUNCTIONS, ZERO RUNTIME DEPENDENCIES

textCaseManipulation
camelCase · snakeCase · titleCase · invertCase

textMaskingAndSecurity
maskEmail · maskPhone · maskCreditCard · moderate

textTransformations
shorten · wordWrap · shuffle · removeDuplicates

textCleaning
trim · normalizeWhitespace · removeWhitespace

textMetadataAndExtraction
URLs · emails · IPs · hashtags · mentions · domains

textSpecializedOperations
levenshteinDistance (DP) · isBalancedBrackets (stack)

textFormatting
Intl API — date · currency · relative time

textAnalysisAndValidation
isPalindrome (⚠ bug: returns string) · matchesPattern

textGeneration
generateRandomString · generateLoremIpsum

import { maskEmail } from 'stringy-core'

tree-shakeable (ESM)

import { _s } from 'stringy-core'

namespace convenience

Pre-commit: Husky + lint-staged → ESLint + Prettier on staged files only. Babel transpiles ESM → CJS for Jest (dev-only, consumers use native ESM).

Key Engineering Decisions

Decision	Chosen	Alternative	Why
Module format	Pure ESM	CommonJS, dual publish	Tree-shaking is the primary value of a utility library; ESM is statically analysable
Zero runtime deps	Yes	lodash, moment.js, etc.	Works in any JS environment; no transitive dependency risk; offline installs cleanly
Formatting	Intl API	date-fns, numeral.js, etc.	Native runtime API — no locale data bundled; locale correct globally; zero added bytes
Test runner	Jest + Babel	Vitest	Vitest is the better modern choice for ESM; Babel/Jest was the decision made at project start
Pre-commit	Husky + lint-staged	CI-only checks	Catches style issues before commit; faster feedback loop than CI
Export patterns	Named + namespace _s	Only one	Named for tree-shaking in production; namespace for REPL/scripting convenience — both documented

Technology Comparisons

ESM vs CommonJS vs UMD

	ESM	CommonJS	UMD
Syntax	import / export	require() / module.exports	IIFE — works as CJS, AMD, or browser global
Tree-shaking	Yes — static analysis	No — dynamic require	No
Node.js support	Native (Node 12+)	Native (always)	Via CJS mode
Standard	Current (ES2015+)	Node.js legacy	Pre-bundler legacy

For stringy-core: pure ESM. Tree-shaking is the primary consumer benefit. A consumer who uses only `maskEmail` should not bundle `levenshteinDistance`. The Jest/Babel config cost is one-time; the tree-shaking benefit is permanent.

Named exports vs namespace vs default export

Named exports: individually tree-shakeable, explicit, bundlers eliminate unused exports.

Namespace `_s`: all functions on one object — not tree-shakeable, but `_s.maskEmail()` is ergonomic for scripting (Lodash-style).

Default export: anti-pattern for utility libraries — ambiguous naming at import site, no tree-shaking.

stringy-core offers both named and `_s`. Consumers choose based on their use case.

Top Interview Q&A

Q: What is the `isPalindrome` bug and how do you fix it?

The function returns the cleaned string instead of a boolean. `isPalindrome('racecar')` returns `'racecar'`, not `true`. The fix is one line: change the return from `return cleaned` to `return cleaned === cleaned.split('').reverse().join('')`. It was caught in documentation review because the JSDoc declares `@returns {boolean}` but the implementation diverges. The bug persists in the published version. Versioning implication: fixing this is technically a PATCH (correcting documented behavior) but if any consumer called `.toUpperCase()` on the return value, it's a breaking change — a MAJOR bump. semver forces this ambiguity to be resolved explicitly.

Q: Why zero dependencies? Couldn't you use an existing library for string operations?

The library is itself a string utility library — depending on another string utility would be circular and unnecessary. More broadly, zero runtime dependencies means: (1) any JS environment works without npm install; (2) no transitive dependency can be yanked, compromised, or cause install failures; (3) the package size is fixed — consumers don't get surprise megabytes. The one place external code is used is the Intl API, which is a native runtime API, not a bundled dependency.

Q: What are contribution stubs?

Functions defined with a complete public signature and JSDoc but with `throw Error('Not implemented')` bodies, marked with `// CONTRIBUTION_STUB`. They define the API surface for a feature without implementing it, telling community contributors exactly what function name, parameters, and return type to implement. This is a structured open-source contribution model — a contributor picks a stub, implements the function, and submits a PR without any API design discussion needed.

Q: Why use the Intl API instead of a library like `date-fns` or `moment.js` for formatting?

`date-fns` is ~40KB; `moment.js` is larger still. The Intl API is provided by every modern JavaScript runtime. For a zero-dependency library, shipping locale data would be contradictory. Intl handles every locale's decimal separators, currency symbol positions, date orderings, and calendar systems natively — it's more correct than any library that ships its own locale data subset. The trade-off is that Intl requires Node 12+ and modern browsers, which is an acceptable constraint in 2025.

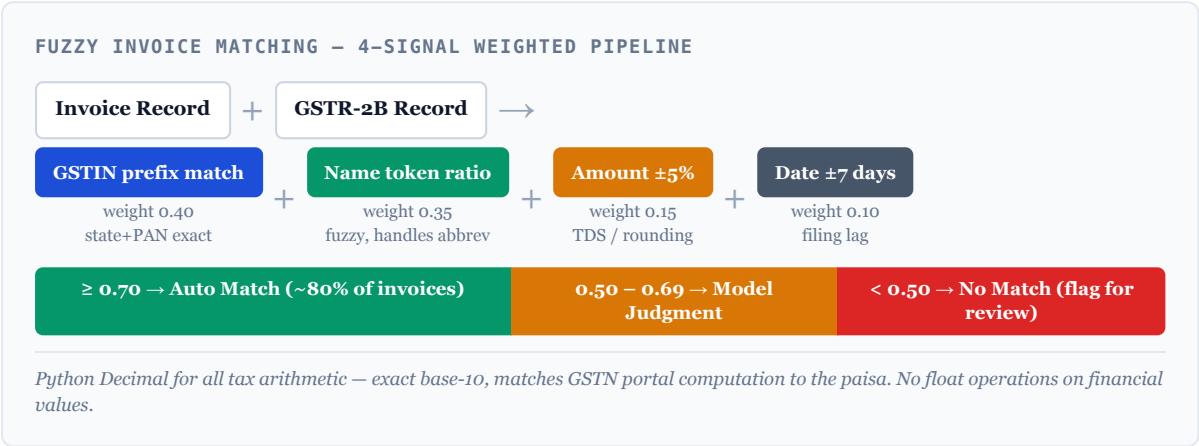
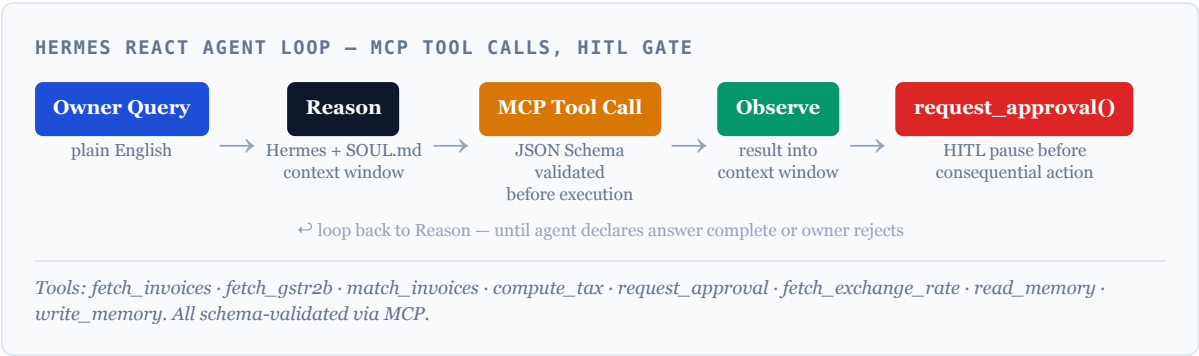
5. Munshi — Sovereign AI GST Reconciliation Agent

Quick Reference

Field	Detail
Client	Bharatvarsh Arts — ₹5Cr (~\$600K) annual revenue
Problem	GST invoice reconciliation against GSTR-2B for ITC claims — was a manual Excel process
Sovereignty	Nothing leaves the machine — hard architectural constraint, not preference
Agent runtime	Hermes + MCP (typed, schema-validated tool calls)
Arithmetic	Python Decimal — exact base-10 computation, no float errors
Storage	SQLite — verdicts, audit trail, persistent memory
HITL	Agent pauses before every consequential action; owner approves or rejects
Stack	Hermes · MCP · FastAPI · Ollama · Python Decimal · Docker Compose · SQLite

Architecture Overview

- Owner asks a question in plain English via chat interface. Hermes receives the query and enters a ReAct loop — reason, call a tool, observe the result, repeat.
- MCP tools** (schema-validated, owner-approved tool list): `fetch_invoices` (reads purchase invoices from local storage), `fetch_gstr2b` (reads GSTR-2B records), `match_invoices` (4-signal fuzzy pipeline), `compute_tax` (Python Decimal arithmetic), `request_approval` (HITL gate), `fetch_exchange_rate` (local cache / forex API), `read_memory` / `write_memory` (SQLite verdicts).
- Fuzzy matching pipeline** — 4 signals, weighted: GSTIN prefix match (exact, weight 0.40), normalized vendor name token set ratio (fuzzy, 0.35), invoice amount within 5% tolerance (0.15), date within 7 days (0.10). Combined score ≥ 0.70 = match. $0.50-0.70$ = model judgment. < 0.50 = no match. Deterministic rules handle ~80%; model reserved for genuine ambiguity.
- Python Decimal** for all tax arithmetic — CGST + SGST + IGST per invoice, aggregated. Integer paise representation. No float operations on financial values.
- HITL:** `request_approval()` tool pauses the agent and presents a plain-English summary to the owner before any consequential action (claiming ITC, recording a verdict, importing an order). Owner approves → agent continues; rejects → agent logs and stops.
- Persistent memory:** approved vendor name resolutions stored in SQLite. Same match judgment is not repeated month-to-month.
- Currency auto-detection:** destination country inferred from the owner's query ("quote for Germany" → EUR). Exchange rate applied automatically. Landed cost shown in EUR and INR equivalent.



Key Engineering Decisions

Decision	Chosen	Alternative	Why
Architecture	Local-first, Ollama	Cloud API (GPT-4)	Financial data sovereignty — invoices, amounts, GSTINs never leave the machine
Tax arithmetic	Python Decimal	float	Exact base-10; matches GSTN portal arithmetic; paisa-level accuracy required
Model role	Judge ambiguous matches only	Compute tax, perform all matching	Deterministic rules handle ~80%; model reserved for genuine judgment calls
HITL timing	Before consequential action	Post-hoc review	Owner verifies before irreversible action — no rollback problem
Agent runtime	Hermes	LangChain AgentExecutor, LangGraph	Open-ended task; MCP schema validation is critical for financial tool calls
Memory scope	SQLite verdicts only	Full conversation history	Raw financial data not persisted; only approved classifications
Overclaiming constraint	SOUL.md hard-bound to available tools	Soft guidelines	Agent must never offer a capability it cannot fulfill

Framework Decision — Hermes vs LangChain vs LangGraph

	Hermes	LangChain	LangGraph
Primary pattern	ReAct loop — AI decides tool path	AgentExecutor / LCEL chains	StateGraph — fixed nodes + edges
Tool protocol	Native MCP — schema validated	Function calling — optional enforcement	Inherits LangChain tools
HITL support	Custom (implement as a tool)	Custom	Built-in <code>interrupt()</code> / resume
Persistence	Bring your own	Bring your own	MemorySaver or DB checkpointer
Best for	Open-ended + MCP enforcement needed	Many integrations, rapid prototype	Known steps + HITL + checkpointing

For Munshi: Hermes. Task is open-ended — model decides tool path dynamically. `request_approval()` can appear at different points depending on the query; LangGraph's fixed graph can't accommodate this. MCP schema validation enforces correct arguments before execution — critical in a financial system.

Decision rule:

- Open-ended, AI decides next step → Agent loop (Hermes or LangChain)
- Need typed MCP enforcement → Hermes
- Many integrations needed → LangChain
- Known steps, fixed in advance → Workflow (LangGraph)
- Need HITL suspend/resume → LangGraph `interrupt()`

Technology Comparisons

float vs Python Decimal

float is IEEE 754 binary floating-point — cannot represent 0.1 exactly in binary. Adding 0.1 + 0.2 in float gives 0.30000000000000004. At scale — hundreds of invoices, each with three tax components — these errors accumulate. Python Decimal uses base-10 arithmetic with configurable precision. `Decimal('0.1') + Decimal('0.2') == Decimal('0.3')` is True. The GSTN portal computes tax in base-10; Munshi must match that arithmetic exactly to avoid filing mismatches. Using float for financial computation is a correctness bug, not a performance trade-off.

Local model vs cloud API for financial data

Cloud API (GPT-4): better model quality, no hardware cost, maintenance-free. But: invoice data, amounts, GSTINs, and vendor names would leave the owner's machine. This is not a privacy preference — it is a hard constraint the client articulated before the project started. There is no cloud-API design for Munshi. Local model quality (qwen2.5 via Ollama) is sufficient for the judgment task: "does this fuzzy match look correct?" is not a frontier reasoning task. The model is a judgment oracle, not the primary reasoner.

Top Interview Q&A

Q: Walk me through a GST reconciliation query end to end.

Owner asks: "How much ITC can I claim this month?" Hermes enters ReAct loop. (1) `fetch_invoices` → reads October purchase invoices from local storage. (2) `fetch_gstr2b` → reads GSTR-2B October records from local file. (3) `match_invoices` → runs 4-signal fuzzy pipeline for each invoice vs GSTR-2B record. Scores ≥ 0.70 auto-matched; $0.50-0.69$ sent to model for judgment; < 0.50 flagged as unmatched. (4) `compute_tax` → Python Decimal arithmetic: CGST + SGST + IGST per matched invoice. (5) `request_approval` → agent presents: "Matched 47 of 52 invoices. Eligible ITC: ₹1,23,456.00. 5 invoices unmatched — details below. Approve to record these verdicts?" Owner approves. (6) `write_memory` → records verdicts in SQLite. Agent responds with final summary. Nothing left the machine.

Q: Why is HITL implemented as an MCP tool rather than a built-in interrupt?

Munshi's approval gate can appear at different points in the tool chain depending on the query. A reconciliation query triggers approval after matching and computation. A landed cost query triggers it after the quote is assembled. If approval were a fixed graph node (LangGraph pattern), it would need to be at a known position. Instead, `request_approval()` is a tool the agent calls whenever it determines it's about to take a consequential action — the agent decides where in its reasoning the gate belongs, not the graph structure. The SOUL.md has explicit instructions to call `request_approval` before irreversible actions; MCP validates the call schema before it executes.

Q: How does the fuzzy matching pipeline work? Why four signals?

GSTIN prefix match (exact, 0.40): GSTINs are structured — first 2 digits are state code, next 10 are PAN. An exact GSTIN match is high confidence. Vendor name token set ratio (0.35): vendor names in invoices ("M/S Raj Trading Co.") often differ from GSTR-2B names ("RAJ TRADING COMPANY") due to abbreviations, case, and punctuation. Token set ratio handles this. Amount within 5% tolerance (0.15): small amount differences occur due to TDS deductions or rounding. Date within 7 days (0.10): invoice date vs GSTR-2B filing date naturally differ. Four signals because any single signal would misfire: same GSTIN but wrong invoice (GSTIN doesn't change across invoices); same name but different vendor; same amount by coincidence. Combined score gives confidence. Model judgment reserved for $0.50-0.69$ range — genuine ambiguity that deterministic rules can't resolve.

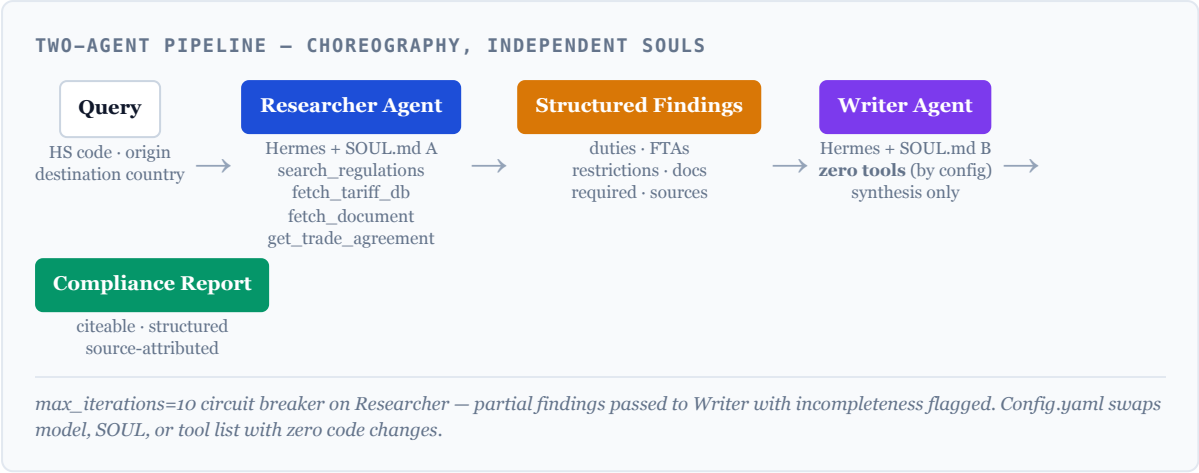
6. Trade Compliance Researcher — Multi-Agent Tariff Research

Quick Reference

Field	Detail
Pattern	Two-agent pipeline: Researcher gathers, Writer synthesises
Runtime	Hermes (wraps any OpenAI-compatible model)
Model (dev)	Ollama + qwen2.5:7b — fully local
Model (prod)	One config line to swap to gpt-4o-mini or any cloud model
Agent identity	Independent SOUL.md files — different instructions, different tool access
Tool protocol	MCP via FastMCP stdio server
Writer constraint	Zero tools — synthesis only, enforced by config
Deployment	Docker Compose: ollama + mcp_tools + hermes_runtime
Key gap	No HITL today; RAGAS eval identified as next investment

Architecture Overview

1. User provides: commodity description, HS code (or partial), origin country, destination country.
2. **Researcher agent** (Hermes + SOUL.md + full MCP tool access): enters ReAct loop. Calls tools iteratively — `search_regulations` , `fetch_tariff_db` , `fetch_document` , `get_trade_agreement` — building a structured findings object. Declares sufficiency when it has covered: basic duty, applicable FTAs, anti-dumping / countervailing duties, import/export restrictions, compliance documents needed.
3. **MCP tools** (FastMCP stdio server, typed JSON Schema): `search_regulations(hs_code, countries, query)` , `fetch_tariff_db(hs_code, origin, destination)` , `fetch_document(url_or_ref)` , `get_trade_agreement(countries)` . All schema-validated before execution.
4. **Writer agent** (Hermes + different SOUL.md + **zero tools**): receives Researcher's structured findings. Synthesises into a citeable compliance report. Cannot call tools — synthesis only. Enforced by Hermes config, not prompt instruction.
5. `max_iterations = 10` circuit breaker on the Researcher. If the loop runs to the limit, partial findings pass to Writer which flags incompleteness.
6. `config.yaml` declares model endpoint, model name, SOUL.md path, and tool allowlist per agent. Swapping model or SOUL requires only a file edit and container restart.



ORCHESTRATION VS CHOREOGRAPHY – WHEN TO USE EACH	
Choreography (this system) <i>Agents coupled only to the artifact format. Either agent upgrades independently. No central controller — resilient to single-node failure.</i> <i>Use when: independent agents, stable handoff schema, no HITL needed between phases.</i>	Orchestration (LangGraph upgrade path) <i>Central StateGraph directs each step. Built-in interrupt() for HITL. MemorySaver checkpoint survives restarts.</i> <i>Use when: HITL needed mid-pipeline, strict ordering required, durable state across time boundaries.</i>

Key Engineering Decisions

Decision	Chosen	Alternative	Why
Two agents	Researcher + Writer	Single agent with all tools	Separation of concerns — research (open-ended) vs synthesis (bounded). Independent SOULs can evolve separately
Writer tools	Zero	Same tools as Researcher	Writer hallucinating tool calls that aren't needed is a real failure mode; zero tools enforced by config
Coordination	Choreography (artifact handoff)	Orchestration (LangGraph StateGraph)	Writer needs only the Researcher's output, not to be embedded in the same control flow; loose coupling
Tool protocol	MCP (FastMCP stdio)	Direct function calls	Typed schema validation; model cannot call with wrong argument types; self-documenting tool interface
Config	config.yaml per agent	Hardcoded in code	Model swap, SOUL swap, tool change = file edit only; zero code changes
Deployment	Docker Compose	Direct process management	Service dependency ordering (mcp_tools health check before hermes starts); reproducible across environments

Technology Comparisons

Orchestration vs Choreography in Multi-Agent Systems

	Orchestration	Choreography
Control flow	Central controller directs each step	Agents react to events/artifacts from each other
HITL integration	Natural — pause points are explicit	Hard — no central place to inject a pause
Debugging	Easy — trace the controller state	Hard — reconstruct causality from event logs
Coupling	Agents coupled to the orchestrator	Agents coupled only to the artifact format
Best for	Known steps, approval gates, audit trails	Independent agents with stable interfaces, parallel pipelines
Examples	LangGraph QuizMe (o1dev), Temporal	This Researcher + Writer pair

For Trade Compliance: choreography. The Writer needs only the Researcher's output artifact — not to be in the same control flow. Independent SOULs mean either agent can be upgraded without touching the other. If a HITL gate between research and synthesis is needed (compliance officer reviews before Writer synthesises), upgrade to LangGraph — one interrupt between phases, MemorySaver checkpoint for durability.

Single agent vs multi-agent

	Single agent	Multi-agent
Context window	One long context — research + synthesis mixed	Each agent has its own clean context
Role confusion	Model must context-switch between researcher and writer roles	Separate SOULs enforce separate identities
Tool access	All tools available throughout	Writer has zero tools — synthesis only; Researcher has all tools
Evolution	Change one thing changes everything	Each agent can be upgraded independently
Debugging	Simpler (one trace)	More complex (two traces, one handoff)

For this system: multi-agent. Research and synthesis are cognitively distinct tasks. Mixing them in one context means the model might start synthesising while it still has outstanding research to do, or call tools during synthesis. Separate agents with separate SOULs enforce clean role separation.

Top Interview Q&A

Q: Why two agents instead of one agent with all the tools?

Two reasons: role clarity and evolution independence. A single agent with research tools and a synthesis mandate will sometimes start writing the report before it's finished researching — the model doesn't have a clean boundary between "gathering phase" and "writing phase." Two agents with separate SOULs enforce this: the Researcher cannot write a report (no synthesis instruction), and the Writer cannot call any tools (zero tool access enforced by config). Evolution independence: we can upgrade the Researcher's SOUL, add a new research tool, or swap its model without touching the Writer's configuration at all.

Q: How does the Writer know it can't call tools — is that a prompt instruction?

No. It's a configuration constraint. Hermes takes a `tools` list in the agent definition. The Writer's `config.yaml` has an empty `tools: []`. Even if the Writer's model emits a tool call (which a well-written SOUL.md prevents), Hermes would reject it because no tools are registered. Config enforcement is stronger than prompt instruction — the model cannot be tricked or confused into calling a tool that isn't registered.

Q: What's the failure mode when the Researcher hits `max_iterations` ?

The Researcher's findings object, however complete or incomplete at iteration 10, is passed to the Writer. The Writer's SOUL.md instructs it to flag incompleteness explicitly in the report: "Anti-dumping duty status could not be fully confirmed within the allowed research depth." The circuit breaker prevents infinite loops. The report is still useful — it shows what was found and what requires manual follow-up. `max_iterations` is configurable in `config.yaml`; a more thorough research run can be triggered by increasing it.

Q: Choreography vs orchestration — when would you switch to LangGraph here?

If a compliance officer needs to review raw findings before the Writer synthesises them, that's a HITL gate between the two agents. LangGraph's `interrupt()` handles this naturally: Researcher returns its findings object → graph pauses → compliance officer reviews → resumes → Writer runs.

MemorySaver checkpoints the state so the pause can last days. Today, the handoff is automatic — no gate, no delay. The moment "human must approve the research before synthesis" becomes a requirement, LangGraph is the right upgrade. The Researcher and Writer would become nodes in a StateGraph instead of independent Hermes agents.

Cross-Project Interview Questions

Q: You use Hermes in Munshi and Trade Compliance, and LangGraph in o1dev. Why both? Can't you just pick one?

They solve different problems. Hermes is a ReAct agent — the model decides what to do next based on what it observes. This is right for open-ended tasks where the tool path isn't known in advance (Munshi's reconciliation queries, Trade Compliance's research depth). LangGraph is a state machine — the steps and transitions are defined before execution starts, with built-in HITL and checkpointing. This is right for known-structure workflows (o1dev's QuizMe: plan → approve → quiz → submit → grade). Using LangGraph for Munshi would mean fighting the graph structure because `request_approval()` appears at different points depending on the query. Using Hermes for QuizMe would mean the agent might skip the approval step because it "decides" to. Tools should match the problem shape.

Q: Multiple projects use SSE. Walk me through when SSE is wrong and when WebSocket is right.

SSE is right when the server streams, the client listens, and the client never needs to send during the stream. This covers: LLM token streaming (o1dev), submission result delivery (CodeMas original architecture). WebSocket is right when the client must send messages while the server is still pushing — a "stop generation" button that cancels the inference mid-stream, or a live collaborative editor where both sides send continuously. In 2025, for LLM streaming specifically: SSE through a standard

HTTP load balancer, no configuration needed. WebSocket requires explicit LB support and a bidirectionality you don't need for streaming.

Q: Three of your projects (o1dev, Munshi, Trade Compliance) run models locally.

What's the real trade-off?

The trade-off is quality vs sovereignty. Cloud models (GPT-4o, Claude 3.5) are significantly more capable, especially on complex reasoning. Local models (qwen2.5:7b, Llama 3.1:8b via Ollama) are good for bounded tasks but weaker on frontier reasoning. The sovereignty argument wins when: the data is legally or commercially sensitive (Munshi — financial data; Trade Compliance — business strategy), the inference cost would be prohibitive at scale, or regulatory compliance requires knowing where data is processed (o1dev — student questions and course content). The architecture in all three cases uses an OpenAI-compatible API — swapping from local to cloud is one environment variable change. Sovereignty is a deployment choice, not a code change.

Q: Each project is a sole-engineer build. What would you add first if you got a team?

- **CodeMas**: a proper CI pipeline with Playwright end-to-end tests for exam submission → result delivery. The system has enough moving parts (SQS, Lambda, Postgres, Redis) that integration tests would catch the failures that unit tests can't.
- **Kalaam**: a language community review board. The bottleneck to adding new languages is not engineering — adding Gujarati is one keyword map entry. The bottleneck is finding native speakers who can validate that the keyword choices are linguistically correct and culturally appropriate.
- **o1dev**: productionise GEPA — wire the eval harness to run automatically on a schedule, automate the diff presentation, and invest in a larger calibration set. The self-improvement loop is the most interesting system in the project and it's currently manual.
- **Munshi**: RAGAS evaluation and HITL for the model judgment step. The fuzzy matching is deterministic and tested; the model judgment for 0.50–0.69 matches is currently unvalidated.
- **Trade Compliance**: HITL between research and synthesis, and structured output from the Researcher (typed findings object vs prose) to reduce Writer hallucination risk.